Avant de coder un module

Un guide pour rédiger son plan Markdown

Support pédagogique • 22 octobre 2026
Objectif : transformer un besoin en un document précis, vérifiable et utilisable par une IA pour implémenter progressivement un module.

Ce que vous devez savoir faire

1. Décrire le besoin et les limites du module.
2. Écrire des règles métier testables.
3. Choisir une solution cohérente avec le projet.
4. Découper le travail en étapes contrôlables.
5. Vérifier le résultat de l'IA avant de poursuivre.

Le document attendu

Un fichier docs/modules/pret-materiel.md, versionné avec le projet. Il contient d'abord le cadrage commun, puis les étapes. Chaque étape indique son objectif, ses prérequis, ses travaux, ses règles, ses livrables, ses tests et ses critères de sortie.

Le fil conducteur

Nous utiliserons un module fictif de prêt de matériel pédagogique : un gestionnaire remet un vidéoprojecteur à un étudiant, puis enregistre son retour. Les choix proposés sont ceux de cet exercice ; ils ne décrivent pas une fonctionnalité existante de RPS Bénin.

Comment utiliser ce support

Pages 2-3 : méthode et trame à reprendre.
Pages 4-7 : exemple concret rempli.
Page 8 : consignes à transmettre à l'IA et contrôle du travail.

Principe essentiel

L'étudiant reste responsable des décisions métier et de la validation. L'IA aide à analyser, implémenter et tester ; une réponse convaincante ne constitue pas une preuve de fonctionnement.

1. Cadrer avant de découper

Partir d’un besoin observable

Évitez « faire un CRUD de prêts ». Écrivez : « Le gestionnaire doit savoir quel matériel est disponible, à qui un exemplaire a été remis et quand il doit revenir. » Décrivez les acteurs, les actions et le résultat attendu.

Séparer quatre types d’information

Objectif : le bénéfice recherché. Exemple : retrouver l’emprunteur d’un matériel.
Règle métier : une condition toujours vraie. Exemple : un exemplaire ne peut pas avoir deux prêts actifs.
Choix technique : le moyen retenu. Exemple : transaction SQL et verrouillage lors du prêt.
Critère d’acceptation : un résultat vérifiable. Exemple : deux demandes simultanées donnent un seul prêt actif.

Examiner le projet existant

Relever la stack et ses versions dans les fichiers du projet ; consulter ses conventions, son authentification, ses permissions, son client HTTP et ses composants. Distinguer « déjà présent », « à créer » et « à confirmer ». Ne pas inventer une table ou une API existante.

Rendre les inconnues visibles

Créer une rubrique « Questions ouvertes » : question, impact, personne qui décide et décision attendue. Une inconnue qui touche les droits, les données ou une règle métier bloque l’étape concernée. Les autres travaux peuvent continuer.

Écrire des règles qui se testent

Trop vague : « sécuriser les prêts ».
Vérifiable : « Un étudiant authentifié ne peut consulter que ses propres prêts ; l’accès au prêt d’un autre étudiant est refusé par l’API. »
Trop vague : « interface ergonomique ».
Vérifiable : « En cas d’échec réseau, le formulaire conserve la saisie et propose une nouvelle tentative. »

Définir une porte de validation

Une étape est terminée quand ses livrables existent, ses tests sont exécutés et ses critères sont vérifiés. Les mentions « prévu », « codé », « testé » et « validé » représentent des états différents.

2. Trame Markdown à réutiliser

En-tête et cadrage commun

Plan d'implémentation - [Nom du module]
1. Identification
Version, auteur, date, statut, validateur.
2. Contexte, problème et objectifs
Utilisateurs concernés, parcours principal, résultat attendu.
3. Périmètre
Fonctionnalités incluses ; exclusions explicites.
4. Existant, stack et contraintes
Versions vérifiées, conventions, dépendances, éléments à créer.
5. Règles métier et autorisations
Règles R01, R02... ; acteurs, actions, périmètres, statuts.
6. Données, API et interface
Champs, contraintes, contrats, écrans et états à gérer.
7. Qualité et risques
Sécurité, concurrence, performance, tests, documentation.
8. Questions ouvertes et décisions
Hypothèses, impacts, décisions validées et leur date.

Bloc à répéter pour chaque étape

Étape N - [Résultat à obtenir]
Statut : à faire / en cours / bloquée / à valider / validée.
Objectif et prérequis
Travaux à réaliser et fichiers concernés
Règles applicables et points de vigilance
Référencer R01... et préciser leur application à cette étape.
Livrables
Tests et cas limites
Critères de sortie
- [ ] Résultat observable à vérifier.
Bilan d'exécution
Fichiers modifiés ; commandes et résultats ; preuves ; limites.
Validation
Validateur, date et décision de passage à l'étape suivante.

Clôture du document

Ajouter une recette globale, les limites connues, les consignes de livraison et un journal des décisions.
Conserver les règles stables en tête ; les référencer dans chaque étape évite des copies contradictoires.

3. Exemple : le prêt de matériel

Cadrage du module

Version 1.0 • Statut : exemple pédagogique à adapter.
Objectif : enregistrer la remise et le retour d'un exemplaire physique, connaître sa disponibilité et conserver son historique.
Acteurs : gestionnaire de matériel ; étudiant emprunteur.

Périmètre retenu

Inclus : liste des exemplaires, création d'un prêt par le gestionnaire, retour, prêts personnels et repérage des retards.
Exclus : réservation, prolongation, amende, notification, gestion des achats et suppression de l'historique. Les comptes et le catalogue sont des prérequis, pas des fonctionnalités à reconstruire ici.

Stack proposée pour cet exercice

Backend Laravel 12 / PHP compatible ; base MySQL 8 ; frontend Vue 3 et TypeScript ; composants PrimeVue. Reprendre les versions exactes du projet de classe et les inscrire dans le document avant le code.
Authentification Web : session sécurisée par cookie HttpOnly et protection CSRF. Réutiliser le dispositif existant après vérification ; ne pas créer une seconde authentification pour ce module.

Règles métier identifiées

R01. Seul le gestionnaire crée un prêt et enregistre un retour.
R02. L'étudiant voit uniquement ses prêts ; le gestionnaire voit ceux de l'établissement. Cet exercice concerne un seul établissement.
R03. Un exemplaire actif possède au maximum un prêt non retourné.
R04. L'emprunteur doit être un étudiant actif ; l'échéance doit être future.
R05. Les dates de remise et de retour sont fixées par le serveur. Un retour déjà enregistré ne crée aucun nouvel événement.
R06. Un prêt conserve son emprunteur, son exemplaire et ses acteurs ; aucune suppression de prêt dans ce module.
R07. Le retard est calculé : échéance dépassée et absence de retour.
R08. Disponibilité = exemplaire actif sans prêt non retourné.

Décisions fixées pour l'exercice

Durée choisie par le gestionnaire, sans plafond dans ce MVP. Plusieurs exemplaires distincts peuvent être prêtés au même étudiant. Pas de modification d'un prêt après création. Stockage des instants en UTC, affichage dans le fuseau configuré de l'établissement. Toute évolution de ces choix doit être décidée et documentée.

4. Contrats et critères d’acceptation

Données proposées, à rapprocher de l’existant

users : identité, statut actif, profil et droits déjà fournis par le projet.
equipment_items : id, inventory_code unique, label, is_active.
loans : id, equipment_item_id, borrower_id, due_at, lent_at, returned_at nullable, created_by, returned_by nullable, timestamps.
Clés étrangères avec suppression restrictive pour les références historiques. Indexer les recherches par emprunteur, exemplaire et échéance. Ne pas stocker un statut « en retard » qui deviendrait périmé.

Intégrité et concurrence

Dans une transaction, verrouiller l’exemplaire, vérifier son activité et l’absence de prêt actif, puis créer le prêt. Prévoir aussi une unicité SQL sur l’exemplaire tant que returned_at est nul : sous MySQL, colonne générée nullable active_item_id et index unique. Vérifier la migration sur la vraie base de test MySQL.
Pour le retour, verrouiller le prêt ; la seconde requête retrouve le même retour sans changer sa date ni son auteur.

Contrat API cible

GET /api/equipment-items : liste paginée, réservée au gestionnaire.
GET /api/loans : liste paginée, limitée au périmètre de l’acteur.
GET /api/loans/{id} : détail autorisé, sinon 404 pour un étudiant tiers.
POST /api/loans : equipment_item_id, borrower_id, due_at ; réponse 201.
POST /api/loans/{id}/return : aucun auteur ni horodatage fourni par le client ; réponse 200, y compris si le retour existe déjà.
401 : session absente ; 403 : action interdite ; 422 : saisie invalide ; 409 : exemplaire devenu indisponible.
Décrire les réponses et exemples dans le contrat API du projet.

Écrans et états obligatoires

Gestionnaire : liste filtrable, formulaire de prêt et détail avec retour à confirmer. Étudiant : liste personnelle et détail en lecture seule.
Prévoir chargement, liste vide, erreur avec réessai, absence de réseau, succès et action en cours. Conserver la saisie en cas d’échec. Après une réponse ambiguë, recharger l’état serveur avant de proposer une nouvelle action.

Trois critères de recette

CA01. Un prêt valide rend l’exemplaire indisponible et apparaît chez le bon étudiant.
CA02. Deux créations concurrentes pour le même exemplaire produisent un seul prêt actif.
CA03. Après retour, l’exemplaire actif redevient disponible et le prêt reste dans l’historique.

5. Étapes de réalisation : le socle

Étape 0 - Vérifier et valider le cadrage

Objectif : rendre le plan applicable au projet réel.
Prérequis : accès au dépôt et au besoin de l'enseignant.
Travaux : relever la stack, vérifier les comptes, le catalogue et les droits ; résoudre les questions ouvertes ; confirmer les contrats de la page 5.
Règles : R01-R08 ; ne pas considérer les prérequis comme déjà disponibles sans vérification.
Livrables : docs/modules/pret-materiel.md et matrice des accès.
Vérification : faire jouer sur papier un prêt valide, un refus et un retour.
Sortie : périmètre, droits, données et décisions validés par l'enseignant.

Étape 1 - Construire les données et autorisations

Objectif : disposer d'un socle intègre et protégé.
Prérequis : étape 0 validée ; utilisateurs et exemplaires disponibles.
Travaux : migrations, modèle Loan, relations, policy et données de test. Emplacements prévus : database/migrations/, app/Models/Loan.php, app/Policies/LoanPolicy.php et tests/Feature/Loans/. Adapter à la structure constatée.
Règles : R01, R02, R03, R06 ; clés étrangères, unicité du prêt actif, liste filtrée côté serveur et détail contrôlé par policy.
Livrables : schéma migrable, autorisations et tests automatisés.
Tests : gestionnaire autorisé ; étudiant refusé en écriture ; prêt d'un tiers inaccessible ; doublon actif refusé ; historique non supprimable.
Sortie : migrations vérifiées sur MySQL de test et tests d'accès réussis.

Étape 2 - Implémenter les opérations et l'API

Objectif : créer un prêt et enregistrer son retour de façon fiable.
Prérequis : étape 1 validée.
Travaux : Form Request, LoanResource, contrôleur fin et service transactionnel justifié par les règles concurrentes ; routes et contrat API. Emplacements : app/Http/Requests/, app/Http/Resources/, app/Http/Controllers/, app/Services/ et routes/.
Règles : R03-R08 ; dates et acteurs calculés côté serveur, validation de l'emprunteur, conflit 409, retour sans double effet.
Livrables : API documentée, factories et tests Feature.
Tests : prêt valide ; étudiant inactif ; échéance passée ; exemplaire inactif ; double création concurrente ; double retour ; utilisateur non authentifié.
Sortie : réponses conformes, CA01-CA03 vérifiés côté backend et tests réussis sur la base cible.

6. Étapes de réalisation : l’usage

Étape 3 - Réaliser les parcours de l’interface

Objectif : permettre les actions métier sans ambiguïté.
Prérequis : API de l’étape 2 vérifiée.
Travaux : types Loan et requêtes typées, service API, liste, formulaire et détail. Emplacements indicatifs : src/types/loan.ts, src/services/loanService.ts, src/views/loans/ et src/components/loans/. Réutiliser le client HTTP et les composants du projet.
Règles : R01, R02, R07, R08 ; le frontend reflète les droits du serveur. Pagination, filtres explicites et libellés métier. Échéance lisible dans le fuseau de l’établissement.
Livrables : parcours gestionnaire et étudiant, états réseau et feedback.
Tests : création puis retour ; erreur 422 ; conflit 409 ; perte réseau ; chargement vide ; navigation clavier et petit écran. Vérifier que l’étudiant ne reçoit pas de prêts tiers dans les réponses réseau.
Sortie : type-check et build réussis ; parcours exercés dans le navigateur ; absence de débordement ou d’action trompeuse.

Étape 4 - Recetter et préparer la livraison

Objectif : prouver que le module répond au besoin global.
Prérequis : étapes 1-3 terminées, environnement de test isolé.
Travaux : exécuter la recette, corriger les défauts, mettre à jour le plan et documenter l’installation.
Règles : R01-R08 ; aucune donnée de démonstration en production ; aucune livraison annoncée sans exécution réelle.
Livrables : rapport de recette, commandes de vérification, limites connues, procédure de migration et de retour arrière.
Tests : scénario complet avec deux comptes étudiants et un gestionnaire ; concurrence sur MySQL ; consultation historique ; contrôle des logs et des données exposées.
Sortie : tous les critères acceptés, défauts bloquants résolus et validation de l’enseignant. Le déploiement est une action distincte qui nécessite une autorisation explicite.

Exemple de bilan à compléter après chaque étape

Statut : à valider.
Fichiers modifiés : [chemins réellement modifiés].
Vérifications : [commande, résultat et date].
Preuves : [rapport de test ou capture du parcours].
Limites : [test non exécuté, dépendance manquante ou anomalie].
Décision : [acceptée / corrections demandées, auteur et date].
Ne jamais remplacer les résultats réels par « tout fonctionne ».

7. Faire travailler l’IA avec le plan

Premier message : analyse du document

« Lis les instructions du dépôt et docs/modules/pret-materiel.md. Vérifie leur cohérence avec le code existant. Identifie les informations manquantes et les conflits éventuels. N’invente aucun champ, endpoint ou comportement existant. Présente les décisions à prendre avant d’implémenter l’étape 0. »

Puis demander une étape précise

« Le cadrage est validé. Implémente uniquement l’étape 1 du plan. Respecte les règles référencées et les conventions existantes. Préserve les modifications sans rapport avec cette tâche. Exécute les tests prévus. Mets à jour le bilan avec les fichiers modifiés, les résultats réels et les limites. N’entame pas l’étape 2 avant ma validation. »

Pour reprendre le travail plus tard

« Relis le plan et son dernier bilan. Compare l’état annoncé avec les fichiers et les tests. Indique ce qui est terminé, incomplet ou bloqué, puis réalise l’étape suivante que j’ai autorisée. Une case cochée ne dispense pas de vérifier les prérequis. »

Travail demandé aux étudiants

Choisissez un module de votre projet et rédigez son fichier Markdown avant le code. Fournissez : un périmètre clair, au moins six règles identifiées, les données et droits, quatre à six étapes, ainsi qu’un test nominal, un refus d’accès et un cas limite pertinents. Faites relire le cadrage avant de demander l’implémentation à l’IA.

Grille d’évaluation proposée / 20

Besoin et périmètre précis : 3 points.
Règles métier et autorisations vérifiables : 4 points.
Cohérence avec la stack et l’existant : 3 points.
Étapes, dépendances et livrables : 4 points.
Tests et critères de sortie : 4 points.
Clarté du Markdown et suivi des décisions : 2 points.

Avant de transmettre votre document

Pouvez-vous expliquer chaque règle sans l’IA ? Chaque étape produit-elle un résultat contrôlable ? Les inconnues sont-elles visibles ? Les erreurs et accès refusés sont-ils prévus ? Les critères permettent-ils à une autre personne de valider le module ?

Origine de la méthode

Structure adaptée des documents du workspace RPS Bénin : « Plan d’implémentation - Module de paie MVP » (docs/plan-module-paie-mvp-2026-07.md, notamment les phases, tests et critères de sortie) et « Module transporteurs » (docs/module-transporteurs-2026-05.md, règles de données, permissions et historique). L’exemple de prêt est une création pédagogique indépendante.